
El algoritmo principal del Monoide de Transición

De un lenguaje a su estructura de álgebra abstracta

Autómatas, expresiones regulares y el monoide de transición $M(A)$

Documento explicativo del proyecto final. Describe, paso a paso y de forma autocontenida, cómo el programa transforma una regla que acepta o rechaza palabras en un objeto algebraico finito, y cómo lee en él la estructura de grupo o de monoide del lenguaje. Pensado para leerse sin conocimientos previos de teoría de autómatas.

Proyecto integrador

Matemática Discreta II & Teoría de la Computación

Marco teórico basado únicamente en dos fuentes:
D. Saracino, *Abstract Algebra: A First Course* (álgebra) y
R. De Castro Korgi, *Introducción a la Teoría de la Computación* (2024).

Índice

1	¿Qué problema resuelve el algoritmo?	1
2	Panorama: el flujo completo	1
3	Preliminares accesibles	1
3.1	Autómata finito determinista (AFD)	2
3.2	De la regex al autómata mínimo (en breve)	2
4	Algoritmo principal, parte 1: construir $M(A)$	2
4.1	La idea: cada palabra es una transformación	2
4.2	El objeto: el monoide de transición	3
4.3	Un detalle de convención (composición izquierda-a-derecha)	3
4.4	La construcción por cierre (BFS)	3
5	Algoritmo principal, parte 2: leer la estructura	4
5.1	La tabla de Cayley	4
5.2	Qué significan estas propiedades	4
6	Un ejemplo completo, de principio a fin	4
7	Fundamento teórico	5
8	La segunda vía: Máquinas de Turing y la frontera de lo regular	6
9	Complejidad y garantías	6
	Referencias	6

1. ¿Qué problema resuelve el algoritmo?

Un **lenguaje** es, simplemente, una regla que separa las palabras en dos grupos: las que *acepta* y las que *rechaza*. Por ejemplo, «las cadenas binarias con un número par de unos» o «las que terminan en 01».

La teoría clásica de autómatas responde a la pregunta «¿cómo reconozco este lenguaje con una máquina?». Este proyecto añade una segunda pregunta, más profunda y de naturaleza algebraica:

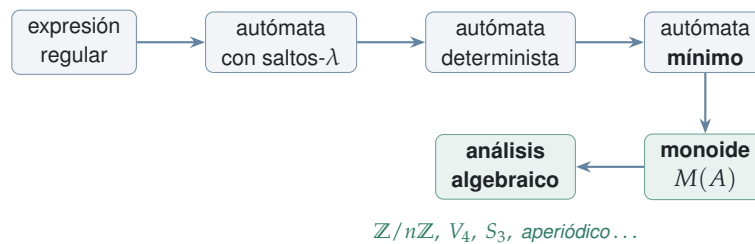
La pregunta central del proyecto

Dado un lenguaje, ¿cuál es su **estructura de álgebra abstracta**? ¿Es un grupo? Si lo es, ¿cuál ($\mathbb{Z}/n\mathbb{Z}$, el grupo de Klein V_4 , el simétrico S_3 ...)? ¿Es abeliano, cíclico? ¿O es un monoide aperiódico?

El puente entre ambos mundos es un objeto llamado el **monoide de transición** $M(A)$, y *construirlo y analizarlo* es el algoritmo principal que describe este documento. Todo lo demás (leer una expresión regular, construir el autómata, minimizarlo) es la maquinaria necesaria para llegar hasta él.

2. Panorama: el flujo completo

El programa encadena cinco etapas. Las tres primeras producen un autómata; las dos últimas son el corazón algebraico.



1. **Parseo.** Leer la expresión regular y entenderla como una estructura jerárquica (un árbol).
2. **Construcción del AFN-λ** (Teorema de Kleene, §2.12). Convertir ese árbol en un autómata con transiciones vacías (λ): fácil de armar, imposible de usar directamente.
3. **Determinización y minimización.** Eliminar la ambigüedad y luego fusionar estados indistinguibles hasta obtener el autómata **mínimo**.
4. **Construcción del monoide $M(A)$.** (*algoritmo principal*)
5. **Análisis algebraico.** (*algoritmo principal*)

Este documento resume las etapas 1–3 (Sección 3) y desarrolla en detalle las etapas 4–5 (Secciones 4 y 5), que son la contribución del proyecto.

3. Preliminares accesibles

3.1 Autómata finito determinista (AFD)

Un AFD es la máquina reconocedora más simple. Formalmente es una quintupla $A = (Q, \Sigma, \delta, q_0, F)$, pero conviene leerla en palabras llanas:

- Q : un conjunto finito de **estados** (los «focos»; en cada instante hay exactamente uno activo).
- Σ : el **alfabeto**, las letras admitidas.
- $\delta : Q \times \Sigma \rightarrow Q$: la **función de transición**, que dado «estado actual» y «letra leída» dice el «estado siguiente». Es total y *determinista*: para cada par hay *una* respuesta.
- $q_0 \in Q$: el estado **inicial**.
- $F \subseteq Q$: los estados **de aceptación** (los «buenos»).

La máquina lee la palabra de izquierda a derecha, aplicando δ letra a letra. Extendemos δ a palabras completas con $\hat{\delta}(q, w) = \text{«estado donde termino si arranco en } q \text{ y leo toda la palabra } w\text{»}$. La palabra w se **acepta** si $\hat{\delta}(q_0, w) \in F$.

3.2 De la regex al autómata mínimo (en breve)

- La **construcción del AFN-λ** (Teorema de Kleene, §2.12) traduce cada operación de la regex (unión, concatenación, estrella de Kleene) en un pequeño autómata con transiciones λ (saltos que no consumen letra), y los pega entre sí.
- La **construcción de subconjuntos** elimina la no-determinación: un *conjunto* de estados del autómata ambiguo se comporta como un *único* estado determinista.

- La **minimización** (refinamiento de particiones) fusiona los estados que son indistinguibles: aquellos desde los cuales *exactamente las mismas palabras futuras* llevan a la aceptación.

Retendremos una sola cosa de esta etapa, pero es crucial:

Por qué el autómata debe ser mínimo

Solo cuando el AFD es **mínimo**, su monoide de transición $M(A)$ es un **invariante del lenguaje** $L(A)$ —depende de L , no de cómo dibujemos la máquina—. Esto lo garantiza el Teorema de Myhill–Nerode de De Castro (Sección 7).

4. Algoritmo principal, parte 1: construir $M(A)$

4.1 La idea: cada palabra es una transformación

Fijemos el AFD mínimo. La observación de partida es sencilla pero poderosa: *cada palabra reordena los estados*. Con precisión, la palabra w induce una función del conjunto de estados en sí mismo,

$$f_w : Q \longrightarrow Q, \quad f_w(q) = \hat{\delta}(q, w),$$

que dice: «si empiezo en el estado q y leo w , termino en $f_w(q)$ ». A una función de Q en Q la llamamos una **transformación** de estados.

Ejemplos de transformación

f_λ (palabra vacía) es la **identidad**: no mueve nada. f_a para una sola letra a es «seguir la flecha a » desde cada estado. Encadenar palabras corresponde a *componer* funciones: $f_{uv} = f_u$ seguida de f_v .

4.2 El objeto: el monoide de transición

Definición 1 (Monoide de transición). El **monoide de transición** de A es el conjunto de todas las transformaciones inducidas por palabras,

$$M(A) = \{ f_w : w \in \Sigma^* \},$$

con la operación de **composición** de funciones y elemento neutro f_λ .

Es un **monoide** porque la composición es asociativa y tiene neutro. Y es **finito**: hay a lo sumo $|Q|^{|Q|}$ funciones de Q en Q , de modo que por más palabras que existan (infinitas), las transformaciones distintas son un número finito. Ese es el milagro que hace posible el análisis:

Compresión de lo infinito

El homomorfismo $\varphi : \Sigma^* \rightarrow M(A)$, $w \mapsto f_w$, colapsa las **infinitas** palabras sobre un conjunto **finito** de transformaciones. Dos palabras caen en el mismo elemento exactamente cuando «hacen lo mismo» a todos los estados. Ese conjunto finito es una huella digital completa del lenguaje.

4.3 Un detalle de convención (composición izquierda-a-derecha)

El programa compone en el orden en que se leen las palabras:

$$(f \cdot g)(q) = g(f(q)).$$

Con esta convención, leer u y luego v es $f_u \cdot f_v$, y por tanto $\varphi(uv) = \varphi(u) \cdot \varphi(v)$: φ es un **homomorfismo** (y no un anti-homomorfismo). Esto es lo que permitirá, más adelante, aplicar limpiamente el teorema de isomorfismo de Saracino (§12).

4.4 La construcción por cierre (BFS)

No se enumeran las $|Q|^{|Q|}$ funciones posibles: se **generan** a partir de las transformaciones de una sola letra, $\{f_a : a \in \Sigma\}$, cerrando bajo composición mediante un recorrido en anchura (BFS).

```

1  Entrada: AFD mínimo  $A = (Q, \Sigma, \delta, q_0, F)$ 
2  Salida:  el conjunto  $M(A)$  de transformaciones distintas
3           (con la palabra mas corta que produce cada una)
4
5   $f_\lambda \leftarrow$  identidad sobre  $Q$ 
6   $M \leftarrow \{f_\lambda\}$            // elementos ya descubiertos
7   $cola \leftarrow [f_\lambda]$        // pendientes por expandir
8  para cada  $a \in \Sigma$ : precomputar  $f_a$  // "seguir la flecha  $a$ "
9
10 mientras la cola no este vacia hacer
11      $f \leftarrow$  sacar frente de la cola
12     para cada letra  $a \in \Sigma$  hacer
13          $g \leftarrow f \cdot f_a$  // componer: leer  $a$  despues de  $f$ 
14         si  $g \notin M$  entonces // transformacion nueva?
15             agregar  $g$  a  $M$ 
16             agregar  $g$  a la cola
17     fin si
18 fin para
19 fin mientras
20 devolver  $M$ 

```

Por qué funciona y por qué termina. Cada elemento nuevo se descubre componiendo uno ya conocido con un generador f_a ; como los generadores producen todo el monoide, el BFS encuentra *todos* los elementos. Y como $|M(A)| \leq |Q|^{|Q|}$ es finito, no pueden aparecer elementos nuevos para siempre: la cola se vacía. Comparar « $g \notin M$ » se hace con una **clave canónica** de cada transformación (la lista ordenada de sus pares $q \mapsto f(q)$), de modo que dos funciones iguales reciben la misma clave. El registro de la «palabra más corta» que genera cada elemento es lo que permite luego decir, por ejemplo, que un grupo cíclico *está generado por la palabra 1*.

5. Algoritmo principal, parte 2: leer la estructura

5.1 La tabla de Cayley

Con los $n = |M(A)|$ elementos ya en mano, se construye su **tabla de composición** (o tabla de Cayley): una matriz $n \times n$ donde la casilla (i, j) guarda el índice de $f_i \cdot f_j$. Es, literalmente, la «tabla de multiplicar» del monoide. A partir de ella, *todas* las propiedades algebraicas son lecturas baratas de la tabla:

Propiedad	Cómo se decide sobre la tabla
Idempotentes	elementos con $f \cdot f = f$ (la diagonal apunta a sí misma).
Unidades $U(M)$	elementos <i>invertibles</i> : existe g con $f \cdot g = g \cdot f = f_\lambda$.
¿Es grupo?	<i>todo</i> elemento es una unidad ($ U(M) = n$).
¿Abeliano?	la tabla es simétrica: $f_i \cdot f_j = f_j \cdot f_i$ para todo par.
Centro $Z(M)$	elementos que conmutan con todos.
¿Cíclico? (grupo)	existe un elemento cuyas potencias sucesivas recorren los n elementos: un <i>generador</i> .
¿Aperiódico?	para todo x , alguna potencia se estabiliza: $x^k = x^{k+1}$ (no hay ciclos no triviales).

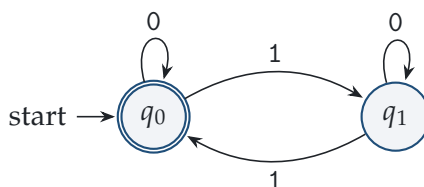
5.2 Qué significan estas propiedades

Aquí es donde Matemática Discreta II y Teoría de la Computación se encuentran:

- Si $M(A)$ es un **grupo**, el programa lo **clasifica**: trivial, $\mathbb{Z}/n\mathbb{Z}$ (cíclico), $V_4 = \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$ (Klein), S_3 , etc. El grupo abstracto es la estructura de conteo del lenguaje.
- Si **no** es grupo pero es **aperiódico**, significa que al iterar cualquier palabra su transformación se estabiliza ($x^k = x^{k+1}$, Saracino §3): $M(A)$ no contiene ningún subgrupo no trivial. El lenguaje solo «detecta patrones», sin contar en ciclos.

6. Un ejemplo completo, de principio a fin

Tomemos el lenguaje $L = \{\text{cadenas binarias con un número par de unos}\}$. Su AFD mínimo tiene dos estados:



con $q_0 = \ll\#unos\ par\gg$ (inicial y de aceptación) y $q_1 = \ll\#unos\ impar\gg$. La función de transición es $\delta(q_0, 0) = q_0$, $\delta(q_0, 1) = q_1$, $\delta(q_1, 0) = q_1$, $\delta(q_1, 1) = q_0$.

Paso 1: las transformaciones. Calculamos qué hace cada letra.

$$f_\lambda = \begin{pmatrix} q_0 \mapsto q_0 \\ q_1 \mapsto q_1 \end{pmatrix}, \quad f_0 = \begin{pmatrix} q_0 \mapsto q_0 \\ q_1 \mapsto q_1 \end{pmatrix}, \quad f_1 = \begin{pmatrix} q_0 \mapsto q_1 \\ q_1 \mapsto q_0 \end{pmatrix}.$$

Observamos que f_0 es la **identidad** (leer un cero no mueve nada), luego $f_0 = f_\lambda$. Y f_1 es el **intercambio** de los dos estados. Componiendo, $f_{11} = f_1 \cdot f_1 = f_\lambda$ (intercambiar dos veces vuelve al inicio). El BFS no encuentra nada más:

$$M(A) = \{e, s\} \quad e = f_\lambda \text{ (identidad)}, \quad s = f_1 \text{ (intercambio)}.$$

Paso 2: la tabla de Cayley.

$\cdot$	e	s
e	e	s
s	s	e

Paso 3: lectura de la estructura.

- **Idempotentes:** solo e (pues $s \cdot s = e \neq s$).
- **Unidades:** ambas ($e^{-1} = e, s^{-1} = s$). Como $|U(M)| = 2 = n$, **es un grupo.** ✓
- **Abeliano:** sí (la tabla es simétrica). **Cíclico:** sí, generado por s , es decir, por la palabra 1. ✓
- **Aperiódico:** no, porque $s^1 = s, s^2 = e, s^3 = s, \dots$ nunca se estabiliza. ✗

Conclusión del ejemplo

$M(A)$ es el grupo cíclico de dos elementos, $M(A) \cong \mathbb{Z}/2\mathbb{Z}$, generado por la palabra 1. Es el mismo objeto matemático que un interruptor de luz o que la suma de paridades (par + impar). La estructura algebraica es la esencia de «contar unos módulo 2».

7. Fundamento teórico

Dos resultados sostienen la corrección y el significado del algoritmo, uno de cada libro del curso. Los enunciamos de forma accesible.

Teorema 1 (Myhill–Nerode; De Castro §2.15). *Un lenguaje L es regular si y solo si la relación «tener el mismo futuro» sobre las palabras tiene un número finito de clases. Esas clases son exactamente los estados del AFD mínimo, que por tanto es único.*

Consecuencia para el algoritmo: al minimizar antes de construir $M(A)$, garantizamos que $M(A)$ es un **invariante del lenguaje L** , y no un artefacto que dependa del dibujo particular de la máquina.

Teorema 2 (Teorema de isomorfismo; Saracino §12). *Para un homomorfismo φ con dominio adecuado,*

$$\Sigma^* / \ker(\varphi) \cong M(A).$$

Es decir: agrupar todas las palabras por «hacen lo mismo» (el núcleo $\ker \varphi$) y quedarse con las clases produce, salvo isomorfismo, el propio $M(A)$. Saracino enuncia este teorema para *grupos* (§12); aquí lo aplicamos, en el mismo espíritu, al homomorfismo de monoides $\varphi : \Sigma^* \rightarrow M(A)$ —esa adaptación es parte del aporte del proyecto—.

Definición 2 (Aperiodicidad; a partir de Saracino §3). *$M(A)$ es **aperiódico** si para todo elemento x alguna potencia se estabiliza: existe k con $x^k = x^{k+1}$. Equivale a que $M(A)$ no contenga ningún subgrupo no trivial.*

La etiqueta «aperiódico» de la página se calcula exactamente con esta definición, usando las potencias de un elemento (Saracino §3); no requiere ninguna fuente fuera de los dos textos del curso.

8. La segunda vía: Máquinas de Turing y la frontera de lo regular

La página ofrece un segundo generador: describes un lenguaje (por ejemplo $a^n b^n c^n$) y se construye una **Máquina de Turing** que lo decide, animando su cinta paso a paso. El vínculo con el álgebra es honesto sobre sus límites:

- Si el lenguaje resulta **regular** (se describe con regex...), se construye su AFD mínimo y se aplica *el mismo* análisis de las Secciones 4 y 5 (Teorema 6.1.1 del libro: todo lenguaje regular es Turing-decible).
- Si el lenguaje **no es regular** (como $a^n b^n$, los palíndromos o ww), no hay AFD finito y su monoide de transición sería **infinito**; el programa lo indica explícitamente y explica por qué la construcción finita no aplica. Muestra, así, la frontera precisa que la teoría predice.

9. Complejidad y garantías

Sea $n = |M(A)|$ el tamaño del monoide y $|Q|$ el número de estados del AFD mínimo.

- **Construcción (BFS):** se generan n elementos; cada composición cuesta $O(|Q|)$ y cada elemento se prueba contra los ya vistos vía su clave canónica. En conjunto, del orden de $O(n |\Sigma| |Q|)$ más el coste de las búsquedas.
- **Análisis (tabla de Cayley):** construirla es $O(n^2 |Q|)$; sobre ella, cada propiedad (grupo, abeliano, centro, idempotentes, aperiodicidad) es un recorrido de $O(n^2)$.
- **Cota y salvaguarda:** $n \leq |Q|^{|Q|}$. Como esta cota crece muy rápido, el programa impone un tope defensivo y, si se supera, informa que el monoide es demasiado grande para construirse en el navegador, en lugar de bloquearse.

El algoritmo en una frase

Cada palabra induce una transformación de los estados; el conjunto finito de transformaciones distintas, cerrado por composición, es el monoide $M(A)$; su tabla de Cayley revela si el lenguaje es, en el fondo, un grupo (y cuál) o un monoide aperiódico. Myhill–Nerode garantiza que ese objeto es *el* invariante canónico del lenguaje.

Referencias

El marco teórico se apoya **únicamente** en estos dos textos:

- **D. Saracino**, *Abstract Algebra: A First Course*, 2.^a ed., Waveland Press, 2008. *Álgebra*: operación binaria y monoide (§1), grupos (§2), potencias / cíclicos / $\mathbb{Z}/n\mathbb{Z}$ y aperiodicidad (§3), productos directos y V_4 (§5, §13), funciones y composición (§6), grupos simétricos S_3 (§7), relaciones de equivalencia (§8), homomorfismos (§11) y teorema de isomorfismo (§12).
- **R. De Castro Korgi**, *Introducción a la Teoría de la Computación*, Universidad Nacional de Colombia, 2024. *Computación*: expresiones regulares (§2.2), AFD/AFN/AFN- λ y Kleene (§2.6–2.13), Myhill–Nerode (§2.15), minimización (§2.16), no regularidad (§3.1) y Máquinas de Turing (§6.1–6.2, Teorema 6.1.1).

El **monoide de transición** $M(A)$ es una construcción propia del proyecto que combina ambas fuentes; no aparece con ese nombre en ninguno de los dos libros.